The calculus in this code

Section 1 — the radial operator

Derived from `docs/guides/calculus_in_this_code.md`, which is the checked source.
This rendering is a derived artifact and can go stale independently of it.

One line of the solver carries the whole geometry:

```
∂c/∂t = (1/r) ∂/∂r (r D_eff ∂c/∂r)
```

Two rs, and neither is a coordinate trick. The one inside is flux times area. The one outside is division by volume. Everything in Part 1 follows from a shell between r and $r + \mathrm{d}r$ whose two faces have different areas, and you can get there without ever writing down a coordinate transformation.

The other four parts are the places where that operator meets the code: why the axis needs no boundary condition, why this solver special-cases it anyway, how a Robin wall becomes one line of ghost-node algebra, and why forty coupled ODEs go to LSODA rather than a fixed step.

Floor: the chain rule, the product rule, and what a partial derivative is. **Ceiling:** ordinary differential equations as objects an integrator solves.

Not here: general curvilinear coordinates, the divergence theorem in 3D, stability proofs, and the sorption terms beyond noting that they set a second timescale.

Each part ends with a *Checks against the source* table. Every row carries a `file:line` and a fragment that line must contain, and `calibration/tests/test_guide_citations.py` verifies all of them — a stale citation fails the suite, and the failure says whether the code moved or the code changed.

Claims in the prose but not in a table are not mechanically verified. Three here: the reading of `bc_coef` as P_{eff} versus k_L, the sorption rates in Part 5, and the stiffness characterisation in Part 5. All three are sourced in the text. None is pinned by a test. A guide implying uniform coverage would be making exactly the claim its own tables exist to discipline.

`README.md`'s *Mathematical framework* states the project's *target formalism*, under its own disclaimer that what the programs integrate differs. This is the other side: what the code integrates. It cross-references those equations rather than restating them, so the two cannot drift into disagreeing.

1. Where the r comes from

The solver's header states the equation it integrates:

```
∂c/∂t = (1/r) ∂/∂r (r D_eff ∂c/∂r)
```

Most courses introduce this as "the Laplacian in cylindrical coordinates" and hand you a table. That presentation makes the r look like bookkeeping from a change of variables. It isn't. It is a statement about area, and you can derive it without ever mentioning coordinates.

Take a shell. Consider the region between radius r and radius $r + \mathrm{d}r$, one unit tall. Two facts about it:

- its **volume** is $2\pi r\,\mathrm{d}r$ (circumference times thickness), and
- its two **faces** have areas $2\pi r$ and $2\pi(r + \mathrm{d}r)$ — *different areas.*

That difference is the whole story. In a slab, the two faces of a slice have the same area, and everything below collapses to $\partial^2 c/\partial x^2$.

Write conservation. The amount of solute in the shell changes at the rate it enters minus the rate it leaves. Rate of flow through a face is flux times area, so with $J(r)$ the flux (amount per area per time, positive outward):

$$\frac{\mathrm{d}}{\mathrm{d}t}\big[\, c \cdot 2\pi r\,\mathrm{d}r \,\big] \;=\; J(r)\cdot 2\pi r \;-\; J(r+\mathrm{d}r)\cdot 2\pi(r+\mathrm{d}r)$$

Divide by the volume $2\pi r\,\mathrm{d}r$:

$$\frac{\partial c}{\partial t} \;=\; -\,\frac{(r+\mathrm{d}r)J(r+\mathrm{d}r) - r\,J(r)}{r\,\mathrm{d}r}$$

The numerator is the change in the product rJ across the shell. Let $\mathrm{d}r \to 0$ and it is by definition the derivative of that product:

$$\frac{\partial c}{\partial t} \;=\; -\frac{1}{r}\frac{\partial (rJ)}{\partial r}$$

Now put Fick's law in. Flux runs down the concentration gradient, $J = -D\,\partial c/\partial r$. Substituting:

$$\frac{\partial c}{\partial t} \;=\; \frac{1}{r}\frac{\partial}{\partial r}\left(rD\frac{\partial c}{\partial r}\right)$$

which is the line in the header, derived from a conservation statement and two areas.

Read the two rs separately, because they are different things. The r *inside* the derivative is flux × area — it is there because a face's area grows with radius. The $1/r$ *outside* is ÷ volume — it is there because you asked for a concentration rather than an amount. Neither is a coordinate artifact.

The physical content is real. Flux converging on a shrinking face concentrates; flux spreading onto a growing face dilutes. A slab has neither effect, which is why its operator has no such term. If you expand the product for constant D:

$$\frac{\partial c}{\partial t} \;=\; D\left[\,\frac{\partial^2 c}{\partial r^2} + \frac{1}{r}\frac{\partial c}{\partial r}\,\right]$$

the first term is ordinary diffusion and **the second is purely geometric** — it is the dilution-onto-a-larger-face effect, and it is proportional to the gradient rather than to its curvature. Note that it carries a $1/r$, which is where Part 2 begins.

Checks against the source

claim	source	must contain
the solver states the cylindrical operator	`biofilms_radiodialysis.R:9`	`(1/r) ∂/∂r (r D_eff ∂c/∂r)`
face weights are area-over-volume at the half-indices	`biofilms_radiodialysis.R:223`	`w_plus <- (r_grid + 0.5 * dr) / r_grid`
and the inward face likewise	`biofilms_radiodialysis.R:224`	`w_minus <- (r_grid - 0.5 * dr) / r_grid`
geometry enters the stencil only through those weights	`biofilms_radiodialysis.R:120`	`(w_plus[i] * (c_vec[i + 1] - c_vec[i]) -`

2. Why the axis isn't a boundary

That $(1/r)\,\partial c/\partial r$ term is singular at $r = 0$. The solution is not.

The resolution is symmetry. The model is radially symmetric — c depends on distance from the axis and on nothing else. A smooth function of radius alone must have $\partial c/\partial r \to 0$ at $r = 0$. If it did not, the profile would have a cusp on the axis, and a cusp in a diffusion profile means a source sitting exactly there. There is no such source in this model.

So at the axis the singular term is $0/0$, and the limit exists. Apply L'Hôpital to the quotient $(\partial c/\partial r)/r$ as $r \to 0$: differentiate numerator and denominator separately, giving $(\partial^2 c/\partial r^2)/1$. The geometric term therefore contributes exactly as much as the ordinary term, and the operator becomes

$$\lim_{r\to 0} D\left[\frac{\partial^2 c}{\partial r^2} + \frac{1}{r}\frac{\partial c}{\partial r}\right] \;=\; 2D\,\frac{\partial^2 c}{\partial r^2}$$

The factor of 2 is not a fudge. It is the geometric term, evaluated in the limit, turning out to equal the ordinary term. The cleanest way to remember it: the factor equals the number of spatial dimensions the radial coordinate is standing in for — 2 here, because r is the radius of a disc in the plane; it would be 3 for a sphere. On the axis, diffusion is that many times as effective at flattening curvature as it is in a slab.

This is why the manuscript calls it symmetry rather than a wall. The condition at $r = 0$ is a *consequence of the geometry*, not a physical boundary anybody chose. There is no membrane on the axis, nothing is sealed, and no flux is being blocked. The zero gradient is what radial symmetry forces, and the "zero-flux" phrasing describes the consequence rather than a mechanism.

Checks against the source

claim	source	must contain
the axis limit is named as L'Hôpital	`biofilms_radiodialysis.R:106`	`L'Hôpital limit`
and implemented with the factor of 2	`biofilms_radiodialysis.R:110`	`dc_dt[1] <- D_eff * 2.0 * (c_vec[2] - c_vec[1]) / dr^2`
the manuscript calls it symmetry, not a wall	`preprint/modeling_radioresistance_and_radiotropic_fitness.tex:741`	`Zero-flux symmetry is imposed at $r = 0$.`

3. Three treatments, three reasons

Here is the part where a description of the *method* and a description of *this code* come apart, and the gap is worth being precise about.

The idealised finite-volume story. Finite difference discretises the **operator** — it approximates $\partial^2 c/\partial r^2$ and $(1/r)\,\partial c/\partial r$ separately with difference quotients, so it meets the $1/r$ head-on and has to special-case the singular node. Finite volume discretises the **conservation law** instead: integrate over each cell and what remains is face fluxes times face areas. You never differentiate $1/r$ at all.

On that telling, both hard parts vanish for free. The innermost cell's inner face sits at radius zero, so its **area** is zero, and the axis condition enforces itself by multiplying a flux by nothing. At the wall, a Robin condition is a statement about flux — and flux is already the quantity the scheme carries — so you set the outermost face flux directly, with no ghost node and no one-sided derivative.

This solver does neither of those things, and the interior is the only place the story holds.

The interior is genuinely flux-form. Geometry enters through two face weights and nowhere else:

```
w_plus  <- (r_grid + 0.5 * dr) / r_grid
w_minus <- (r_grid - 0.5 * dr) / r_grid
```

Those are face area over cell volume, exactly the ratio Part 1 derived, evaluated at the half-indices $r \pm \mathrm{d}r/2$ where the faces are. The stencil then reads as (outward face flux − inward face flux), weighted, and no $1/r$ is ever differentiated.

The axis is special-cased. The innermost node does not get a zero-area face; it gets the L'Hôpital limit from Part 2 written in directly:

```
dc_dt[1] <- D_eff * 2.0 * (c_vec[2] - c_vec[1]) / dr^2 + ...
```

That line is the one Part 2's table already pins, so it is not repeated below. What the table here adds is the second half of the evidence: the face weights at that node are set to `NA` on purpose, so any future code that reaches for a metric weight there fails loudly instead of silently using one.

The wall uses a ghost node. Not a directly-set face flux — the eliminated-ghost algebra that the idealised account says finite volume avoids:

```
c_ghost <- c_vec[Nr - 1] -
  2.0 * dr * bc_coef * (c_vec[Nr] - c_ext) / D_eff
```

So the code is a hybrid, and the contrast is what makes it worth explaining. Flux-form in the interior, L'Hôpital at the axis, ghost-node elimination at the wall — three treatments and three reasons. The interior needs no special case *because* it is flux-form; the two boundaries need one *because* the scheme is not carried through to the faces there. The idealised account is correct about the method and it explains why the middle looks the way it does. It is not a description of the two ends.

None of that contradicts the manuscript. §5 describes the axis limit and the Robin condition as represented explicitly in the semi-discrete operator, and both are: the L'Hôpital limit and the ghost elimination are written out inside the right-hand side rather than imposed on it from outside. "Explicitly in the operator" and "through the face structure" are different claims, and only the second is the one this part is distinguishing from.

Checks against the source

claim	source	must contain
the axis node's face weights are deliberately unusable	`biofilms_radiodialysis.R:226`	`w_plus[1] <- NA_real_; w_minus[1] <- NA_real_`
the wall uses a ghost node, not a set face flux	`biofilms_radiodialysis.R:132`	`c_ghost <- c_vec[Nr - 1] -`
the scheme is named finite-volume method of lines	`biofilms_radiodialysis.R:32`	`finite-volume method of lines`
the manuscript describes both ends as handled in the operator	`preprint/modeling_radioresistance_and_radiotropic_fitness.tex:993`	`represented explicitly in the semi-discrete operator`

4. The Robin condition

At the outer wall the solver imposes:

$$-D_{\text{eff}} \left.\frac{\partial c}{\partial r}\right|_{r=R} = P_{\text{eff}}(t)\,\big(c(R,t) - c_{\text{ext}}\big)$$

Read it as a flux balance. The left side is the diffusive flux arriving at the wall from inside. The right side is what the membrane will pass, proportional to the concentration difference across it, with P_{eff} the constant of proportionality — a permeability, in units of velocity.

It interpolates between the two conditions you already know.

- $P_{\text{eff}} \to \infty$: the right side can only stay finite if $c(R) \to c_{\text{ext}}$. The membrane is invisible; this is a Dirichlet condition.
- $P_{\text{eff}} \to 0$: the right side vanishes, so $\partial c/\partial r|_R = 0$. Nothing crosses; this is a sealed wall, a Neumann condition.

The membrane is the only thing in this model that is neither, and that is exactly why the condition has to be Robin rather than one of the two simpler kinds.

Getting from that statement to the code is one substitution. Introduce a ghost node $c[N_r{+}1]$ just outside the domain and approximate the wall derivative with a centred difference, which is second-order:

$$\left.\frac{\partial c}{\partial r}\right|_R \approx \frac{c[N_r{+}1] - c[N_r{-}1]}{2\,\mathrm{d}r}$$

Put that into the Robin condition and solve for the ghost:

$$-D\,\frac{c[N_r{+}1] - c[N_r{-}1]}{2\,\mathrm{d}r} = \texttt{bc\_coef}\,\big(c[N_r] - c_{\text{ext}}\big)$$

$$c[N_r{+}1] = c[N_r{-}1] - 2\,\mathrm{d}r\;\texttt{bc\_coef}\,\big(c[N_r] - c_{\text{ext}}\big)/D$$

which is the line in the solver. The ghost is then substituted into the ordinary interior stencil, so it never appears in the state vector — it exists only inside the arithmetic for the last node.

One naming caution, and the code is emphatic about it. `bc_coef` is P_{eff} in the cylindrical geometry and k_L, an external liquid-film mass-transfer coefficient, in the slab. They are different physical quantities, and the slab's must not be reported as a permeability. The stencil is written to take whichever the producer declares.

Checks against the source

claim	source	must contain
the condition as stated	`biofilms_radiodialysis.R:20`	`-D_eff ∂c/∂r|_{r=R} = P_eff(t) · (c(R,t) − c_ext)`
the ghost substitution, second line	`biofilms_radiodialysis.R:133`	`2.0 * dr * bc_coef * (c_vec[Nr] - c_ext) / D_eff`

5. Method of lines, and why LSODA

The construction. Discretise space and leave time alone. Forty cells, each with a concentration, gives forty coupled ordinary differential equations — plus forty more for the sorbed phase and one for membrane integrity, packed into a single state vector of length $2N_r + 1$. That is the method of lines, and the point of it is that once space is discretised you are holding an ODE system, and ODE integrators are a solved problem you can hand it to.

Why not step it forward yourself? An explicit step is limited by stability, not by accuracy. The diffusion operator's eigenvalues scale as $D/\Delta r^2$, and an explicit Euler step is stable only while $|1 + \Delta t\,\lambda| \le 1$ for every eigenvalue λ — which caps the step at roughly $\Delta r^2/2D$. Halve the grid spacing and the allowed step drops by four.

That bound is not hypothetical here: the Julia port's explicit stepper substeps against exactly it, with a safety factor:

```
dt_stable = 0.4 * dr^2 / (2.0 * rd.params.D_eff)
```

That is only one of the timescales. Sorption runs at its own rate — an uptake of $U = 0.056\ \mathrm{s}^{-1}$ against a relaxation of $1/(k_{\mathrm{des}} + k_{\mathrm{loss}}) = 166.7$ s. When the fastest process in a system forces a step far shorter than the slowest process needs for accuracy, the system is **stiff**, and that is the whole of the definition.

Whether *this* system is stiff is not computed anywhere in this repository, and this guide does not assert it. Stiffness is a ratio, and no artifact here reports one: `analysis/verify_radiodialysis_stability.py` computes the explicit-Euler bound and the conservative real-axis limit, but not a stiffness ratio against the sorption timescale. The two rates quoted above are the ingredients such a ratio would be built from, and they are themselves among the prose claims no test pins. So the honest statement is that LSODA is a defensible default for a system with two visibly separated timescales, not a measured requirement — and closing that gap means shipping the ratio from a producer, in the shape the rest of this repository already uses.

That is what LSODA is for. It monitors the solution and switches between an Adams method (cheap, for the non-stiff stretches) and BDF (implicit, stable at large steps, for the stiff ones), choosing its own step size to meet the tolerances it is given rather than a stability limit you computed by hand.

A caveat this repository is careful about, and so is this guide. The explicit bound above is analysed in `analysis/verify_radiodialysis_stability.py`, which separates three claims that are easy to conflate — continuous stability, semi-discrete stability, and explicit-Euler stability — and computes the conservative real-axis limit directly. Its conclusion about the Julia path is that the explicit stepper is stable at $\mathrm{d}t = 0.5$ only because r is in lattice units while D_{eff} is in $\mathrm{cm}^2\,\mathrm{s}^{-1}$, which is the documented units error behind `RADIODIALYSIS: BLOCKED`. **A stability margin that comes from a units mismatch is not evidence the step is safe**, and none of the reasoning in this part should be read as saying the coupled path is sound.

Checks against the source

claim	source	must contain
forty cells by default	`biofilms_radiodialysis.R:230`	`default_parms <- function(Nr = 40, R = 1.0)`
the state vector packs c, s and m	`biofilms_radiodialysis.R:50`	`y[1 .. Nr] = c_i`
LSODA is the integrator	`biofilms_radiodialysis.R:400`	`method = "lsoda"`
the Julia port substeps against the diffusion bound	`biofilms_potts.jl:1453`	`dt_stable = 0.4 * dr^2 / (2.0 * rd.params.D_eff)`

claim	source	must contain
the explicit-Euler limit is computed, not asserted	`analysis/verify_radiodialysis_stability.py:153`	`conservative_real_axis_limit`

Every check-table row is verified by `calibration/tests/test_guide_citations.py`: the file must exist, the line must exist, and it must contain the fragment. A stale line number fails the suite, and the failure message distinguishes "the code moved" from "the code changed." Long paths are broken across lines at underscores; the breaks are typographic only.